CSC-201: Computer Organization & Architecture

Autumn 2026

Processor Design Assignment

1. Introduction

This document gives the instruction set architecture (ISA) of RISC201, the processor we are designing for the CSC-201 assignment. It covers the register set, the instruction formats, the full instruction list with opcodes and meanings, the addressing modes, how a condition is tested and the memory model the programs run in. The rest of our design (the 4-stage and 6-stage pipelines, the horizontal microprogrammed control unit and the ALU) is built on top of this.

RISC201 is a 32-bit load-store RISC machine. Every instruction is one 32-bit word, and only the load/store and stack instructions (LD, ST, PUSH and POP) access data memory; all other instructions work on registers. We kept the ISA small on purpose: one instruction width, six formats and only the addressing modes we actually need. With a team of our size this keeps the number of cases that the decoder, control unit, assembler and simulator have to handle manageable, while still letting us write proper test programs with arithmetic (including multiply, divide and remainder), memory access, stack-based function calls and conditional branches.

Section 2 is the reference for the ISA. It fixes the opcode, format and meaning of every instruction, so that the assembler, the microcode and the ALU can be written in parallel by different people against the same specification. Sections 3 and 4 describe the two pipelines, Section 5 compares them and Section 6 describes the control unit.

2. The RISC201 Instruction Set Architecture

2.1 Registers

RISC201 has 16 general-purpose registers, R0 to R15, each 32 bits wide. There are also three special registers (PC, SP and LR) that are outside the register file and cannot be named in the rd, rs1 or rs2 fields. PUSH and POP reach LR through a dedicated bit of the S format (Section 2.2), not through a register field. There is no condition-code register; Section 2.5 says why.

Register	Width	Encoding	Description
R0	32 bits	0000	Always reads as 0, and writes to it are ignored. Useful as a constant zero and as a destination when a result should be thrown away.
R1 – R15	32 bits each	0001 – 1111	General-purpose. The ISA does not reserve any of them for a special use; any register convention (for example for function arguments) is decided by the assembler and software, not by the hardware.
PC	32 bits	not encoded	Program counter. Holds the address of the instruction being fetched. Goes up by 4 every cycle unless a taken branch, B, CALL or RET changes it. Reset to 0.
SP	32 bits	not encoded	Stack pointer for a full-descending stack (decremented before a push, incremented after a pop). Used implicitly by PUSH and POP. Reset to the top of memory (Section 2.9).
LR	32 bits	not encoded	Link register. CALL writes the return address here and RET reads it. PUSH and POP can also name it through the S-format L bit, which is how a nested or recursive call saves it.

2.2 Instruction Formats

Every instruction is 32 bits long. Bits [31:26] are always the 6-bit opcode, so there is room for 64 opcodes; we use 38 of them and leave 27 free for later. How the other 26 bits are split depends on the format. There are six formats:

Format	Fields (MSB → LSB)	Used by	Notes
R (Register)	opcode[31:26] rd[25:22] rs1[21:18] rs2[17:14] unused[13:0]	ADD, SUB, MUL, DIV, MOD, AND, OR, XOR, NOT, SHL, SRL, SRA, FADD, FSUB, FMUL, FDIV, FEQ, FGT, ITOF, FTOI	Three register operands. The low 14 bits are unused. NOT, ITOF and FTOI take only two, and encode rs2 as 0.
I (Immediate)	opcode[31:26] rd[25:22] rs1[21:18] imm[17:0]	ADDI, SUBI, MULI, DIVI, MODI, LD, ST, MOV, MOVHI	One 18-bit sign-extended immediate or displacement.
S (Stack)	opcode[31:26] rs1[25:22] L[21] unused[20:0]	PUSH, POP	One register field plus the L bit, which selects LR instead; SP is implied.
B (Branch)	opcode[31:26] rs1[25:22] rs2[21:18] offset[17:0]	BEQ, BGT	Compares rs1 and rs2; 18-bit PC-relative offset.
J (Jump/Call)	opcode[31:26] offset[25:0]	B, CALL	No registers; 26-bit PC-relative offset for long jumps.
N (No operand)	opcode[31:26] unused[25:0]	RET, NOP	The opcode alone is enough.

Some points that apply to all formats:

- Register fields are 4 bits, one for each of R0–R15.
- The 18-bit immediate or offset in I-type and B-type instructions is sign-extended to 32 bits during decode.
- The 26-bit offset of B and CALL is also sign-extended, which gives a range of about ±33.5 million bytes. This is much more than any of our test programs will need.
- ST has no destination register, so the rd position holds the register to be stored. In the same way, POP puts its destination register in the rs1 position of the S format (PUSH and POP never need two registers).
- Bit 21 of the S format is the L bit. L = 0 uses the rs1 field as usual; L = 1 ignores it and makes LR the source of PUSH or the destination of POP. LR is outside the register file and no register field can name it, so this bit is the only way a nested or recursive CALL can save and restore its return address. L = 0 in every other encoding, so nothing else changes. Write it as `PUSH LR` or `POP LR`; the assembler encodes the bit and leaves the rs1 field zero.
- NOT (no rs2) and MOV (no imm) leave one field unused. The assembler always fills unused fields with 0.
- MUL, DIV and MOD are encoded like ADD, and MULI, DIVI and MODI like ADDI, with sext(imm) as the second operand. For example, MUL R3, R1, R2 is 000011 0011 0001 0010 000…0 = 0x0CC48000, and MULI R3, R1, −5 is 001111 0011 0001 11111111111111011 = 0x3CC7FFFB.

2.3 Complete Instruction Set

The table below is the full opcode map. For each instruction it gives the format, the opcode in binary and hex and the operation. Opcodes 0x25–0x3F are unused and kept for later. We grouped the opcodes by type so that related instructions are next to each other: 0x00 does nothing, 0x01–0x05 register arithmetic, 0x06–0x0C logical and shift, 0x0D–0x11 immediate arithmetic, 0x12–0x15 memory and moves, 0x16–0x17 stack, 0x18–0x1C branches, jumps and calls, and 0x1D–0x24 floating point (Section 2.10). There is no halt instruction: a program ends when the PC walks past the last word of its code (Section 2.9), so the opcode space is spent entirely on work.

Mnemonic & Operands	Fmt	Opcode (bin)	Opcode (hex)	Operation	Description
NOP	N	000000	0x00	PC ← PC + 4	Does nothing. Used for padding and scheduling.

Mnemonic & Operands	Fmt	Opcode (bin)	Opcode (hex)	Operation	Description
ADD rd, rs1, rs2	R	000001	0x01	rd ← rs1 + rs2	32-bit two's complement addition. Overflow wraps silently: there is no bit to record it (Section 2.5).
SUB rd, rs1, rs2	R	000010	0x02	rd ← rs1 − rs2	Done as rs1 + (~rs2) + 1, wrapping on overflow like ADD.
MUL rd, rs1, rs2	R	000011	0x03	rd ← (rs1 × rs2)[31:0]	Signed 32 × 32 multiply. Only the low 32 bits of the 64-bit product go to rd (they are the same for signed and unsigned inputs). A product too wide for 32 bits is truncated, not reported.
DIV rd, rs1, rs2	R	000100	0x04	rd ← rs1 ÷ rs2 (signed, rounded towards 0)	Signed division. If rs2 = 0 a divide-by-zero exception is raised (Section 2.7). $(-2^{31}) \div (-1)$ is the one quotient too large for 32 bits, and rd then gets 0x80000000.
MOD rd, rs1, rs2	R	000101	0x05	rd ← rs1 − (rs1 ÷ rs2) × rs2	Remainder of the same division as DIV. The result is 0 or has the same sign as rs1. rs2 = 0 raises a divide-by-zero exception.
AND rd, rs1, rs2	R	000110	0x06	rd ← rs1 AND rs2	Bitwise AND.
OR rd, rs1, rs2	R	000111	0x07	rd ← rs1 OR rs2	Bitwise OR.
XOR rd, rs1, rs2	R	001000	0x08	rd ← rs1 XOR rs2	Bitwise XOR. XOR rd, rd, rd is the usual way to clear a register.
NOT rd, rs1	R	001001	0x09	rd ← ~rs1	One's complement of rs1. The rs2 field is unused (0000).
SHL rd, rs1, rs2	R	001010	0x0A	rd ← rs1 << rs2[4:0]	Logical left shift by the low 5 bits of rs2 (0–31). Bits leaving bit 31 are discarded.
SRL rd, rs1, rs2	R	001011	0x0B	rd ← rs1 >> rs2[4:0] (logical)	Logical right shift by rs2[4:0]: zeros are shifted in and bits leaving bit 0 are discarded.
SRA rd, rs1, rs2	R	001100	0x0C	rd ← rs1 >> rs2[4:0] (arithmetic)	Arithmetic right shift by rs2[4:0]: copies of bit 31 are shifted in, so the sign is kept.
ADDI rd, rs1, imm	I	001101	0x0D	rd ← rs1 + sext(imm)	Adds an 18-bit sign-extended immediate. Also used as the second instruction of LI (Section 2.6).
SUBI rd, rs1, imm	I	001110	0x0E	rd ← rs1 − sext(imm)	Subtracts an 18-bit sign-extended immediate.
MULI rd, rs1, imm	I	001111	0x0F	rd ← (rs1 × sext(imm))[31:0]	Same as MUL, but the second operand is the sign-extended immediate.
DIVI rd, rs1, imm	I	010000	0x10	rd ← rs1 ÷ sext(imm) (signed, rounded towards 0)	Same as DIV, but divides by the sign-extended immediate. The assembler does not accept imm = 0.
MODI rd, rs1, imm	I	010001	0x11	rd ← rs1 − (rs1 ÷ sext(imm)) × sext(imm)	Same as MOD, with the sign-extended immediate as the divisor. The assembler does not accept imm = 0.
LD rd, imm(rs1)	I	010010	0x12	rd ← Mem[rs1 + sext(imm)]	Loads a 32-bit word from rs1 + offset (up to ±131,072 bytes from rs1). The address must be word-aligned, or a memory exception is raised (Section 2.9).
ST rs2, imm(rs1)	I	010011	0x13	Mem[rs1 + sext(imm)] ← rs2	Stores a 32-bit word at rs1 + offset. rs2 sits in the field that other I-type instructions use for rd, since ST does not write a register.
MOV rd, rs1	I	010100	0x14	rd ← rs1	Copies one register to another. The imm field is unused (0).
MOVHI rd, imm	I	010101	0x15	rd[31:14] ← imm[17:0]; rd[13:0] ← 0	Sets the upper 18 bits of rd and clears the lower 14. Used with ADDI to build 32-bit constants (Section 2.6).
PUSH rs1	S	010110	0x16	SP ← SP − 4; Mem[SP] ← rs1	Full-descending push: SP is decremented by 4 first, then rs1 is stored. With L = 1 the stored value is LR instead.
POP rd	S	010111	0x17	rd ← Mem[SP]; SP ← SP + 4	Full-descending pop: load first, then SP is incremented by 4. rd is in the same field that PUSH uses for rs1, or LR when L = 1.

Mnemonic & Operands	Fmt	Opcode (bin)	Opcode (hex)	Operation	Description
CALL offset	J	011000	0x18	LR ← PC + 4; PC ← PC + 4 + sext(offset)	PC-relative subroutine call. The return address is saved in LR.
RET	N	011001	0x19	PC ← LR	Returns from a subroutine that was entered with CALL.
B offset	J	011010	0x1A	PC ← PC + 4 + sext(offset)	Unconditional PC-relative jump. The 26-bit offset reaches much further than the conditional branches.
BEQ rs1, rs2, offset	B	011011	0x1B	PC ← (rs1 == rs2) ? PC+4+sext(offset) : PC+4	Branch if equal. rs1 and rs2 go to a comparator in the decode stage; there are no condition flags to test (Section 2.5).
BGT rs1, rs2, offset	B	011100	0x1C	PC ← (rs1 > rs2, signed) ? PC+4+sext(offset) : PC+4	Branch if rs1 > rs2 (signed). BLT is done by swapping the operands (Section 2.6).
FADD rd, rs1, rs2	R	011101	0x1D	rd ← rs1 + rs2 (binary32)	Single-precision addition, rounded to nearest even. What the FPU reports goes to FPSR (Section 2.10).
FSUB rd, rs1, rs2	R	011110	0x1E	rd ← rs1 − rs2 (binary32)	The same adder with the sign bit of rs2 inverted.
FMUL rd, rs1, rs2	R	011111	0x1F	rd ← rs1 × rs2 (binary32)	Significands multiply, exponents add. Raises overflow or underflow when the result leaves the representable range.
FDIV rd, rs1, rs2	R	100000	0x20	rd ← rs1 ÷ rs2 (binary32)	Unlike integer DIV, a zero divisor does not raise an exception: the result is a correctly signed infinity and FPSR records divide-by-zero (Section 2.10).
FEQ rd, rs1, rs2	R	100001	0x21	rd ← (rs1 == rs2) ? 1 : 0	Writes an *integer* 0 or 1, so the result is branched on with BGT rd, R0 (taken when true) or BEQ rd, R0 (when false). +0 and −0 compare equal; any NaN compares false.
FGT rd, rs1, rs2	R	100010	0x22	rd ← (rs1 > rs2) ? 1 : 0	The only ordered comparison with an opcode. Less-than is the same instruction with the operands exchanged, which the assembler spells FLT (Section 2.6).
ITOF rd, rs1	R	100011	0x23	rd ← binary32(int32(rs1))	Signed integer to float. Magnitudes above 2^{24} have more bits than the significand holds, so the result is rounded and FPSR records inexact. rs2 is unused (0000).
FTOI rd, rs1	R	100100	0x24	rd ← int32(rs1), truncated toward zero	Float to signed integer. This is the one operation that does not round to nearest even. Out of range it saturates and raises invalid (Section 2.10). rs2 is unused (0000).

2.4 Addressing Modes

Mode	Used by	Description
Register-direct	ADD, SUB, MUL, DIV, MOD, AND, OR, XOR, NOT, SHL, SRL, SRA, MOV, PUSH/POP, BEQ/BGT	The operand is the value of a register named in the instruction.
Immediate	ADDI, SUBI, MULI, DIVI, MODI, MOVHI	The operand is a constant stored in the instruction. For all of these except MOVHI it is sign-extended to 32 bits; MOVHI places it in the upper 18 bits instead.
Base + displacement	LD, ST	Effective address = register value + sign-extended 18-bit immediate, i.e. up to about ±131,072 bytes from the base register.
PC-relative	B, BEQ, BGT, CALL	Target = PC + 4 + sign-extended offset, i.e. relative to the instruction after the branch.
Implicit	PUSH/POP (SP, and LR when L = 1), RET (LR)	The address or target comes from a special register that is not named in a register field of the instruction.

We did not include a separate register-indirect mode (LD rd, 0(rs1) does the same job), auto-increment/decrement addressing (apart from the SP update in PUSH and POP) or register + register indexed addressing. These can be added later if needed, but leaving them out keeps the decoder and the effective-address part of the microcode small.

2.5 Condition Flags (None)

RISC201 has no condition-code register. No instruction sets a zero, negative, carry or overflow bit, and no instruction reads one. Arithmetic wraps silently at 32 bits, a multiply keeps only the low half of its product, and the one arithmetic condition the machine reacts to at all is a zero divisor, which is an exception rather than a flag (Section 2.7).

The conditional branches do the work a flags register normally does. BEQ and BGT carry both of their source registers to a comparator in the decode stage and redirect the PC on its answer, so the comparison and the branch that uses it are one instruction and there is nothing to keep between them. The floating-point comparisons reach the same place from the other side: FEQ and FGT write an integer 0 or 1 into rd, which BEQ or BGT then branches on (Section 2.10).

We chose this over condition codes deliberately. A flags register is one location that nearly every arithmetic instruction writes and every conditional branch reads, so a pipelined implementation has to forward and interlock it exactly like a register operand while it stays invisible in the instruction encoding. Comparing registers in the branch removes that dependency altogether, which is why the RISC designs this ISA follows do the same. The cost is that multi-word arithmetic has to recover a carry explicitly, with a shift and a compare, instead of chaining an add-with-carry; we judged that a fair price for the instruction mix in Section 5.

The floating-point status register FPSR is not a condition-code register and is unaffected by any of this: it records IEEE exceptions that an instruction has already reported in its result, and no branch tests it (Section 2.10).

2.6 Pseudo-Instructions

The assembler expands the following pseudo-instructions into real instructions from Section 2.3. None of them has its own opcode.

Pseudo-instruction	Expands to	Notes
LI rd, imm32	MOVHI rd, imm32[31:14]; ADDI rd, rd, imm32[13:0]	Loads any 32-bit constant. ADDI sign-extends its immediate, so if bit 13 of imm32 is 1, a plain split would give wrong upper bits. In that case the assembler adds 1 to the upper part and encodes the lower part as a negative number, the same trick as the lui/addi pair in RISC-V, so the two instructions together give exactly imm32.
FLI rd, float	MOVHI rd, bits[31:14]; ADDI rd, rd, bits[13:0]	The same pair as LI, with the 32 bits taken from a decimal float literal: FLI R1, 3.14159 loads 0x40490FD0. Turning the text into those bits is the assembler's job (Section 2.10); it is not something the processor does.
BLT rs1, rs2, offset	BGT rs2, rs1, offset	Branch if less than (signed), done by swapping the two operands.
BGE / BLE rs1, rs2, offset	two conditional branches to the same target (e.g. BGE: BEQ rs1, rs2, target then BGT rs1, rs2, target)	There is no single opcode for greater-or-equal or less-or-equal, so these take one extra instruction. The second branch sits 4 bytes further on, so the assembler encodes its offset 4 smaller to reach the same target.
CLR rd	XOR rd, rd, rd	Sets a register to zero.
FLT rd, rs1, rs2	FGT rd, rs2, rs1	Floating-point less-than, done by exchanging the operands, exactly as BLT does for BGT. This is exact for every input including NaN, which NOT FEQ AND NOT FGT is not (Section 2.10).
PUSHM r1, r2, …	PUSH r1; PUSH r2; …	Saves a list of registers, LR included, in the order written.

Pseudo-instruction	Expands to	Notes
POPM r1, r2, …	… POP r2; POP r1	Restores the same list, written in the same order but popped in reverse. The full-descending LIFO discipline of Section 2.12 is therefore the assembler's job rather than the programmer's: POPM takes the identical operand list to the PUSHM it undoes.

MULI, DIVI and MODI use the same 18-bit sign-extended immediate as ADDI and SUBI, so the constant must be between −131,072 and +131,071. For a larger constant, load it with LI first and then use MUL, DIV or MOD. The assembler gives an error for DIVI or MODI with an immediate of 0. If such an instruction ends up in memory anyway (for example from hand-written hex), the hardware still raises the divide-by-zero exception of Section 2.7. Multiplying or dividing by a power of two can also be done with the shifts, but neither right shift is the same thing as a division: SRL is logical, so it matches DIV/DIVI only when the dividend is not negative, and SRA keeps the sign but rounds towards −∞ while DIV rounds towards zero, so the two differ for a negative dividend that does not divide exactly.

2.7 Multiply, Divide and Modulus

Like SimpleRisc in Sarangi's textbook, RISC201 has multiply, divide and remainder as normal ALU instructions that take either a register or an immediate as the second operand, instead of writing results into separate HI/LO registers as MIPS does. All six instructions treat their operands as signed 32-bit two's complement numbers. In this section, rs2 (or "the divisor") means rs2 for MUL, DIV and MOD and sext(imm) for MULI, DIVI and MODI; the rules are the same for both.

- **MUL and MULI.** The product of two 32-bit numbers can need 64 bits, but only the low 32 bits are written to rd. The low half is the same whether the inputs are treated as signed or unsigned, so these instructions also work for unsigned multiplication. Nothing reports that the result was cut off. RISC201 has no condition codes (Section 2.5), so a program that needs to know whether a product overflowed 32 bits has to check the operands before multiplying.
- **DIV and MOD** (and DIVI and MODI) give the two results of the same division. The quotient q is rounded towards zero, and the remainder r satisfies rs1 = q × rs2 + r with |r| < |rs2|, where r is either 0 or has the same sign as rs1. This matches / and % in C and Java.
- **Overflow.** The only overflow case is $(-2^{31}) \div (-1)$, because the answer $+2^{31}$ does not fit in 32 bits. DIV and DIVI then write the wrapped 0x80000000, which is the architectural answer and not an exception. MOD and MODI write 0, the correct remainder.
- **Division by zero.** If the divisor is 0, a divide-by-zero exception is raised in the execute stage. The instruction does not write rd, and it and every instruction behind it in the pipeline are squashed, and the simulator stops and prints the exception with the PC of the instruction that caused it (Section 6.1).

Some examples we used to check these rules:

rs1 (dividend)	rs2 (divisor)	DIV result	MOD result	Check: q × rs2 + r
7	2	3	1	3 × 2 + 1 = 7
−7 (0xFFFFFFF9)	2	−3 (0xFFFFFFFD)	−1 (0xFFFFFFFF)	−3 × 2 + (−1) = −7
7	−2 (0xFFFFFFFE)	−3 (0xFFFFFFFD)	1	(−3) × (−2) + 1 = 7
−7 (0xFFFFFFF9)	−2 (0xFFFFFFFE)	3	−1 (0xFFFFFFFF)	3 × (−2) + (−1) = −7
0x80000000 (-2^{31})	−1 (0xFFFFFFFF)	0x80000000	0	overflow case
any	0	exception	exception	divide by zero

rs1	rs2	True product	rd (MUL result)	Note
−3 (0xFFFFFFFD)	5	−15	0xFFFFFFF1 (−15)	exact in 32 bits
0x00010000	0x00010000	2^{32}	0x00000000	low 32 bits only; the product needed 33
0x7FFFFFFF	2	$2^{32} - 2$	0xFFFFFFFE	low 32 bits only

2.8 The Multiply/Divide Unit (MDU)

This part is not strictly part of the ISA, since a program can only notice it through timing, but the pipeline sections depend on it, so we describe it here. From now on we call MUL, DIV, MOD, MULI, DIVI and MODI the **MDU instructions**. They are executed by a separate multiply/divide unit (MDU) placed next to the ALU in the execute stage (EX/MEM in the 4-stage pipeline, EX in the 6-stage pipeline). The same divider hardware produces both the quotient and the remainder, and ALUOp selects which one is used (Section 6.2). An immediate form differs from its register form only in where the second operand comes from (ALUSrcB = 01), so both take the same time.

An MDU instruction stays in the execute stage for **L** cycles, where L depends on the algorithm. The algorithms below are the ones from the computer arithmetic chapter of Sarangi's book. We build four of them in full, written as the iteration the hardware performs rather than as a call on the host machine's arithmetic: both multipliers and both dividers. Their latencies are therefore not figures we picked but properties of those loops, and the simulator takes them from the unit itself. The `pipe` command selects which pair runs, so Section 5 measures the comparison between them instead of asserting it. A fifth setting, `default`, is not an algorithm: it hands the operation to the host machine's own operator in a single cycle and drives no adder, which makes it the floor the real units are measured against.

Unit	Algorithm	Latency L (execute-stage cycles)	Notes
Multiplier	Iterative shift-and-add	32	Works on the magnitudes of the operands and adds one partial product per cycle; the sign is fixed at the end. Its adder count is therefore the population count of |rs2|. The baseline Booth is measured against, since the two take the same 32 cycles and differ only in adder operations.
Multiplier	Booth (radix-2), iterative	32	Works directly on two's complement numbers, so it needs no magnitude step and no special case for -2^{31}. An add or subtract is only needed where two neighbouring multiplier bits differ (01 or 10), so a run of 1s needs fewer adder operations than shift-and-add, but it still takes 32 cycles.
Divider	Restoring	32 + k, where k ($0 \le k \le 32$) is the number of restore steps	Each of the 32 iterations subtracts the divisor. If the partial remainder becomes negative, one extra cycle adds the divisor back, so the time depends on the data. A trial succeeds exactly where the quotient takes a 1 bit, so k is the number of 0 bits in the quotient and L is 64 − popcount(quotient). The only algorithm here whose L is not a constant, which is why the pipelines ask the unit for the cycle count of each individual operation rather than holding a figure for the instruction.
Divider	Non-restoring	33	32 add-or-subtract iterations with no restore steps, plus one last cycle to correct a negative remainder. We always spend this last cycle so that L is fixed. The same iteration yields the quotient and the remainder, and ALUOp picks between them.
Either	`default` (host operator)	1	Not an iteration, and not an implementable unit: the operation is handed to the host machine's `*` or `/` and finishes in one cycle, driving no adder. It is here so that the cost of the real algorithms can be read against a floor, which is how Section 5.1 uses it.

The non-restoring divider needs unsigned inputs, so the MDU takes the magnitudes of the operands in the first cycle and applies the signs in the last: the quotient sign is sign(rs1) XOR sign(rs2), and the remainder gets the sign of rs1. This does not add any cycles. The divide-by-zero check is done in the first execute cycle, before the iterations start, so a zero divisor costs one cycle rather than a whole division.

Booth's advantage over shift-and-add does not appear in L at all, because both take 32 cycles. It appears in how often the adder runs, and the simulator counts that: a multiplier of all 1s is one run of 1s and drives the adder once, where shift-and-add would drive it 32 times, while a multiplier of alternating bits reaches the worst case of 32. The count is reported as `mdu_adder_ops`, and Section 5 uses it to measure the difference the cycle column cannot show. Only the multiplier operand is recoded, so the count depends on the bit pattern of rs2 and not of rs1, even though the product of course does not.

Which pair of algorithms the unit runs is chosen with the `pipe` command: `pipe 4` on its own is the machine as built, Booth and non-restoring, while `pipe 4 shift-and-add restoring` swaps both. The names it accepts are `booth`, `shift-and-add` and `default` for the multiplier and `non-restoring`, `restoring` and `default` for the divider. `default` is not an algorithm: it hands the operation to the host machine's own arithmetic in a single cycle and drives no adder, which is the combinational bound the four iterative algorithms are read against in Section 5. Whichever pair is selected, the result in rd is the same; only the cycles and the adder count move.

2.9 Memory Model and Program Image

RISC201 has one flat, byte-addressable memory holding both code and data (a von Neumann layout), 64 KB by default. The simulator reads the size from the `RISC201_MEM_KB` environment variable at start-up, so a program that needs more can ask for it.

Instruction fetch, LD, ST, PUSH and POP all move whole 32-bit words and all require the address to be a multiple of 4. A misaligned address, or one past the end of memory, raises a memory exception rather than wrapping or reading rubbish (Section 6.1). RISC201 has no byte load or store, so the byte order inside a word is never architecturally visible; the simulator stores words little-endian. On reset the PC is 0, every general register is 0, and SP is the size of memory, i.e. one word past the highest valid address, which is what a full-descending stack wants: the first PUSH writes the top word.

A program image is therefore just the contents of memory from address 0 upwards. The assembler emits one: the `.text` section first, starting at 0, and the `.data` section immediately after the last instruction. The assembler writes it as a `.hex` file, one 32-bit word per line in hexadecimal, or as a `.bin` file when invoked with `--bin`, which is the same words as raw bytes, most significant byte first. The simulator loads either, choosing by the file's extension.

A program ends when the PC reaches the first address past its code. This is why RISC201 needs no halt instruction and spends no opcode on one. The two sections share a memory but not a purpose, so the simulator has to know where the split is: the assembler writes the size of `.text` as a `; code <n>` header line at the top of the `.hex` file, and the loader passes it on. Without that figure the data would be fetched and executed as instructions. A `.bin` image carries no header and is taken to be code throughout. An exception stops the machine the same way, but records the fault; the debugger tells the two apart by whether an error was recorded. Assembly source uses `name:` labels, `;` to start a comment, and five directives:

Directive	Meaning
.text	The lines that follow are code. This is the default at the start of a file.
.data	The lines that follow are data, placed after the last instruction.
.word v1, v2, …	One 32-bit word per value. A value may be a number or a label, which assembles to its address.
.float v1, v2, …	One binary32 word per value, written in decimal (Section 2.10).
.space n	n words of zero. Only allowed in .data.

Two further directives, `.macro` and `.endm`, are handled by the preprocessor before either pass and never reach the sections above (Section 2.13).

A branch target has to be a whole number of instructions away, so the assembler rejects an offset that is not a multiple of 4, one that does not fit in the format's 18 or 26 signed bits, or a label that lives in `.data`.

2.10 Floating Point

RISC201 holds a single-precision float in one 32-bit word, in the IEEE-754 binary32 layout: one sign bit, eight exponent bits and twenty-three mantissa bits.

Field	Bits	Width	Meaning
S (sign)	31	1	0 positive, 1 negative. This is sign-magnitude, not two's complement, which is why a signed integer BGT cannot order floats and FGT has to exist.
exponent	30:23	8	Excess-127: a stored 127 means 2^0. The values 0x00 and 0xFF are reserved for the special cases below.
mantissa	22:0	23	The fraction. A normal number carries an implicit leading 1, so twenty-three stored bits give twenty-four bits of precision.

What a word means depends on the exponent field:

Exponent	Mantissa	Value
0x00	0	±0. The sign bit still distinguishes −0 from +0, and FEQ reports them equal.
0x00	≠ 0	±0.mantissa × 2^{-126}, a *subnormal*. There is no implicit leading 1, which is what lets values keep shrinking below the smallest normal, losing precision instead of simply becoming zero.
0x01–0xFE	any	±1.mantissa × $2^{(\text{exponent} - 127)}$, a *normal* number.
0xFF	0	±infinity.
0xFF	≠ 0	NaN, Not a Number.

The largest finite value is about 3.4028235×10^{38}, the smallest normal about $1.1754944 \times 10^{-38}$ and the smallest subnormal about 1.4×10^{-45}. As an example, 3.14159 is sign 0, exponent 10000000 (128, so 2^1) and mantissa 10010010000111111010000, which is the word 0x40490FD0.

Rounding. Arithmetic rounds to nearest, ties to even, and there is no rounding-mode register to change that. FTOI is the single exception: it truncates toward zero, matching what a cast does in C. A NaN produced by the FPU is always the same word, 0x7FC00000, so a NaN payload never propagates.

Getting a float into a register. There is no floating-point immediate, because the I format has only eighteen bits of immediate and a binary32 needs thirty-two. Values arrive three ways: ITOF converts whatever an integer instruction computed, LD reads a word a `.float` directive placed in memory, and the FLI pseudo-instruction (Section 2.6) builds a literal with MOVHI and ADDI. Nothing marks a register as holding a float rather than an integer; there is one register file and the opcode decides how its bits are read.

Comparisons. FEQ and FGT write an integer 0 or 1 into rd rather than setting a flag, so the branch unit needs no floating-point knowledge at all: `BGT rd, R0, target` is taken when the comparison was true and `BEQ rd, R0, target` when it was false. Two opcodes are enough; every relation is reachable from them with the integer logic instructions:

Relation	Built from
a == b	FEQ
a != b	FEQ, then branch with BEQ rd, R0 instead of BGT

Relation	Built from
a > b	FGT
a < b	FGT with the operands exchanged, which the assembler spells FLT (Section 2.6)
a ≥ b	FGT OR FEQ
a ≤ b	FLT OR FEQ

Why less-than is an operand swap and not a negation. Trichotomy says that for ordered values exactly one of <, == and > holds, which tempts one to write a < b as NOT FEQ AND NOT FGT and drop another opcode. That is wrong, and NaN is why: every comparison against a NaN is false, so both negations are true and the conjunction claims a < b when the two values are not ordered at all. Repairing it needs an unordered test (`FEQ rd, a, a` is 0 only for a NaN) and costs several instructions. Exchanging the operands of FGT costs nothing and is exact for every input, which is the same trade the integer ISA already makes with BLT and BGT.

For related reasons there is no FLE or FGE opcode: the OR above is correct even for NaN, because every term is then 0. There is no FNEG, FABS or FMOV either: XOR with 0x80000000 flips the sign, AND with 0x7FFFFFFF clears it, and MOV already copies a word.

Exceptions. The five IEEE conditions are reported in FPSR, a sticky status register: the FPU sets a bit and only a reset clears it. None of them stops the machine, which is the point – a program that raised inexact on almost every instruction would never finish. This is a real difference from integer arithmetic, where DIV by zero raises the exception of Section 2.7 and halts.

Flag	Raised by	Result delivered
Invalid	inf − inf, inf × 0, inf ÷ inf, 0 ÷ 0, FTOI of a NaN or out of range	NaN, or the saturated integer for FTOI
divide-by-Zero	FDIV with a finite non-zero numerator and a zero divisor	±infinity, signed by the operands
Overflow	a result too large to represent	±infinity (inexact is raised as well)
Underflow	a subnormal result that was also inexact	the subnormal, or ±0
ineXact	the rounded result differs from the exact one	the rounded value

FTOI saturates rather than wrapping. A NaN, +infinity or any value at or above 2^{31} gives 0x7FFFFFFF; −infinity or anything below -2^{31} gives 0x80000000; all four raise invalid. Exactly -2^{31} is representable and converts without a flag. A value between −1 and 1 truncates to 0, so FTOI of −0.5 is 0, not −1.

2.11 The Floating-Point Unit (FPU)

The FPU sits beside the ALU in the execute stage and shares its result bus, the same arrangement the MDU has (Section 2.8). The control word’s FpuEn line decides which of the three units reads the operands, and ALUOp then says which operation (Section 6.2). Everything is done on integer fields: unpack the sign, exponent and mantissa, restore the implicit leading 1, operate, normalize, round once at the end, and pack.

Three extra bits are carried below the significand throughout – guard, round and sticky. The lowest is sticky in the literal sense: once a discarded bit was non-zero it stays non-zero, however far the value is later shifted. That is what lets a single rounding step at the end see the whole exact result it is rounding away from, instead of rounding twice and drifting.

Operation	How it is done	Latency
FADD, FSUB	Align the smaller exponent's significand, add or subtract, then normalize. Subtraction of near-equal values cancels leading bits and needs a long left shift to renormalize, which is the slow path.	4
FMUL	Multiply the two 24-bit significands into a 48-bit product and add the exponents.	5
FDIV	Divide the significands, with an exact remainder test supplying the sticky bit, and subtract the exponents.	14
FEQ, FGT	A compare of the raw words after NaN and ±0 are handled, so no arithmetic is needed.	1
ITOF, FTOI	A priority encode to find the leading bit, then a shift: no adder is involved.	2

These latencies are fixed, unlike the MDU's, which the simulator takes as a parameter because which multiplier and divider to build is a design choice this document compares. Each FPU operation has one algorithm. Like the MDU, the FPU holds the execute stage for its latency and freezes the stages behind it; the debugger shows the progress as `EX: 0x1D FADD [FPU 2/4]`.

2.12 The Stack and its Boundaries

RISC201 has a **full-descending** stack: SP addresses the last word pushed, and the stack grows towards lower addresses. `PUSH` pre-decrements (SP ← SP − 4, then store at the new SP) and `POP` post-increments (load from SP, then SP ← SP + 4), so the two are exact inverses. SP is initialised to the top of memory, which is the *stack base*: the address one word above the first word any program can push.

Nothing else in the machine moves SP. There is no SP-relative addressing mode, so `LD` and `ST` cannot reach a stack slot by offsetting SP; a routine reads its own frame by popping it. That restriction is what makes the three checks below decidable in hardware.

Stack boundaries

The stack is the one region the datapath protects at run time. Section 2.9 lays the program image out from address 0 upwards (`.text` then `.data`), and the stack descends from the top, so the free space between them belongs to neither. The first word past the image is the *stack limit*; the loader takes it from the image size and the simulator enforces it:

Condition	Raised by	Meaning
`stack overflow`	PUSH writing below the stack limit	The stack has grown into `.data` or `.text`. Usually runaway recursion or a `PUSH` with no matching `POP`.
`stack underflow`	POP with SP at the stack base	A `POP` with nothing pushed, which would read whatever lies above the stack.
`stack corruption`	ST to an address at or above SP	A plain store reaching inside a live frame. No legitimate program does this, because SP is not an addressing base.

All three are checked in the memory stage, *before* the access, and all three report through the same path as a misaligned or out-of-range access (Section 2.7): the instruction writes nothing, SP does not move, and the machine stops with the fault named. They are not maskable, and a correct program never meets them. Because the check sits in the shared execute semantics, the 4-stage and 6-stage pipelines inherit it unchanged.

Call frames and the L bit

A frame is created by convention, not by hardware: RISC201 has no frame pointer and no `ENTER` instruction. What marks a frame is the saved return address, and LR can only reach the stack through the S-format L bit of

Section 2.2:

```
FACT:   BGT  R1, R0, RECURSE     ; base case
        LI   R2, 1
        RET
RECURSE:
        PUSH LR                  ; the CALL below overwrites LR
        PUSH R1                  ; the local
        SUBI R1, R1, 1
        CALL FACT
        POP  R1
        POP  LR
        MUL  R2, R2, R1
        RET
```

The callee pushes LR first, so the saved return address is the highest-addressed word of its frame, and the locals sit below it. Nothing in memory distinguishes a saved LR from a saved GPR, so the simulator records the address as the `PUSH LR` executes and drops it again when SP rises past it. That record is what the debugger's `stack` command uses to cut the stack into frames:

```
risc201> stack
  base 0x00010000   SP 0x0000FFF0   limit 0x00000048  (program image end)
  4 word(s) in use, 2 frame(s), 65448 byte(s) before overflow
                  +-------------------------------------+  base  frame 1
  0x0000FFFC      | 0x00000008  saved LR -> 0x00000008  |
  0x0000FFF8      | 0x00000005  5                       |
                  +-------------------------------------+  frame 2
  0x0000FFF4      | 0x00000030  saved LR -> 0x00000030  |
  0x0000FFF0 SP ->| 0x00000004  4                       |
                  +-------------------------------------+  free space down to the image end
```

2.13 Macros and the Preprocessor

The assembler runs a **preprocessor** over the source text before pass 1. It takes `.macro` definitions out of the file and replaces every invocation with the definition's body, so the two assembly passes never learn that macros exist and a macro can expand to anything the assembler already accepts, including a pseudo-instruction or another macro.

```
.macro ENTER saved              ; a callee's prologue
        PUSHM LR, \saved
.endm

.macro LEAVE saved              ; ... and its epilogue, same list
        POPM  LR, \saved
        RET
.endm

SQUARE: ENTER R4
        MUL   R5, R4, R4
        LEAVE R4
```

Construct	Meaning
.macro NAME p1, p2, …	Begins a definition. The parameter list may be empty. A macro must be defined before it is used, and its name may not be that of a real instruction or a pseudo-instruction.
.endm	Ends the definition. Everything between the two lines is the body, kept as text.
`\p`	In the body, the operand passed as parameter `p`. An invocation must supply exactly as many operands as the definition names.
`\@`	A number unique to each expansion. A body that defines a label needs it (`POS\@:`), or the second invocation would define the same label twice.

A macro may invoke another macro. Expansion stops at a depth of 32, which is how a macro that invokes itself is caught: the preprocessor has no way to see a base case, because the branch that would end the recursion is not evaluated until the program runs. A label written on an invocation line attaches to the first instruction the body produces.

Errors name both ends of the expansion. A line that came out of a macro is reported at the invocation, with the macro and the body line in brackets, because either end may hold the mistake:

```
Error, line 19 (ABS, macro line 12): bad register 'R99' (use R0 to R15, or LR with PUSH/POP)
```

`PUSHM` and `POPM` (Section 2.6) are built into the assembler rather than supplied as macros, because they take a register list of any length and the macro parameters above are positional. They are what a prologue and epilogue are written in terms of.

3. The 4-Stage RISC201 Pipeline

The 4-stage pipeline has the stages Instruction Fetch (IF), Decode/Operand Fetch (DE/OF), Execute/Memory Access (EX/MEM) and Writeback (WB). Combining decode with the register read, and execute with memory access, means we only need three pipeline registers, but two of the stages have more work to do and therefore a longer delay.

The control unit is horizontal and microprogrammed (Section 6). Its control word is read once per instruction in DE/OF and passed down the pipeline latches, and each later stage uses the fields it needs.

3.1 Stage Definitions

Stage	Name	What happens
1	IF (Instruction Fetch)	IR ← Mem_instr[PC] and PC ← PC + 4. If the instruction in DE/OF in the same cycle redirects the PC (taken branch, B, CALL or RET), the new target is used for the next fetch instead. Only the instruction memory port is used.
2	DE/OF (Decode & Operand Fetch)	The opcode is decoded and the control word is read (Section 6). The fields are extracted (Section 2.2), the 18-bit immediate or offset is sign-extended, and rs1 and rs2 are read from the register file. For BEQ/BGT the two register values go to a separate comparator, so the branch outcome and target are known by the end of this cycle. B and CALL do not use registers, so their target is computed directly from the offset, also in this cycle. RET sets the PC to the current value of LR, also in this cycle. For CALL, PC + 4 is latched here and carried down the pipeline like an ALU result so that it can be written to LR in WB. An illegal opcode also takes effect here: it stops the fetch and squashes whatever is behind it.
3	EX/MEM (Execute & Memory Access)	The ALU computes the result of an R-type or I-type instruction, or the address rs1 + sext(imm) for LD/ST, and in the same cycle the data memory is read (LD, POP) or written (ST, PUSH). Instructions that do not use memory just pass the ALU result on. MDU instructions go to the MDU instead of the ALU (MDUEn = 1), stay in this stage for L cycles (Section 2.8) and do not use memory. Floating-point instructions do the same at the FPU (FpuEn = 1), for the latencies of Section 2.11. This stage has the most control signals active at once, because the ALU and memory controls are both used here.
4	WB (Writeback)	The ALU/MDU result, the loaded word, or the saved PC + 4 (for CALL) is written to the destination: rd for most instructions and LR for CALL, selected by RegDst.

3.2 Instruction Timing

Case 1: independent instructions. They overlap with one stage between them:

Cycle	1	2	3	4	5
ADD r3,r1,r2	IF	DE/OF	EX/MEM	WB	
LD r4,0(r5)		IF	DE/OF	EX/MEM	WB

Case 2: dependent ALU instructions, no stall. SUB needs r3 from the ADD just before it. ADD's result is ready at the end of its EX/MEM stage (cycle 3), and SUB only needs it when it enters EX/MEM in cycle 4. One forwarding path, from the EX/MEM→WB latch back to the ALU input mux in EX/MEM, gets the value there in time:

Cycle	1	2	3	4	5
ADD r3,r1,r2	IF	DE/OF	EX/MEM	WB	
SUB r6,r3,r4		IF	DE/OF	EX/MEM	WB

Case 3: load-use, also no stall. The classic 5-stage pipeline needs a stall here, but ours does not. The address calculation and the memory read both finish inside LD's EX/MEM stage (cycle 3), so the loaded value is ready at the same point as an ALU result would be. ADD enters EX/MEM in cycle 4, the same distance as in Case 2, so the same forwarding path works:

Cycle	1	2	3	4	5
LD r3,0(r1)	IF	DE/OF	EX/MEM	WB	
ADD r6,r3,r4		IF	DE/OF	EX/MEM	WB

This follows from merging EX and MEM. In the 5-stage pipeline (and in our 6-stage one, Section 4) a loaded value is ready one stage later than an ALU result, and that difference is what causes the load-use stall. Here there is no difference, so one forwarding path covers both cases without any extra hardware.

Case 4: taken branch, one bubble. BEQ/BGT, B, CALL and RET are all resolved by the end of DE/OF (stage 2). The instruction fetched behind them in cycle 2 is squashed, and the target is fetched from the next cycle:

Cycle	1	2	3	4	5	6
BEQ r1,r2,L	IF	DE/OF	EX/MEM	WB		
(squashed)		IF	squashed			
L: ...			IF	DE/OF	EX/MEM	WB

Case 5: branch operand, one stall. The comparator sits in DE/OF, one stage before any result appears, so the forwarding path of Case 2 is too late for it. A second path feeds the comparator from EX/MEM, and a conditional branch whose operand is produced by the instruction directly in front of it waits one cycle for that path. RET behaves the same way when the instruction in front of it writes LR. A branch whose operands are older than that costs nothing, and B and CALL never wait because they read no register:

Cycle	1	2	3	4	5	6
SUB r3,r1,r2	IF	DE/OF	EX/MEM	WB		
BEQ r3,r0,L		IF	DE/OF	stall	EX/MEM	WB
(squashed)			IF	IF	squashed	
L: ...					IF	DE/OF

Case 6: MDU instructions, L − 1 stalls. An MDU instruction stays in EX/MEM for L cycles. The pipeline is in-order and has only one EX/MEM stage, so the next instruction has to wait in DE/OF (and the one after it in IF) until the MDU finishes. That is L − 1 stall cycles, whether or not the next instruction uses the result. The result is ready at the end of the last EX/MEM cycle, so the normal EX/MEM→WB forwarding path passes it on without

any further stall. We drew the table with L = 4 so that it fits on the page; with the default Booth multiplier L is 32. ADD would normally enter EX/MEM in cycle 4, but here it enters in cycle 7:

Cycle	1	2	3	4	5	6	7	8
MUL r3,r1,r2	IF	DE/OF	EX/MEM	EX/MEM	EX/MEM	EX/MEM	WB	
ADD r6,r3,r4		IF	DE/OF	stall	stall	stall	EX/MEM	WB

3.3 Hazards and Limitations

- **Structural:** LD and ST need the ALU and the data memory in the same cycle in EX/MEM. This makes the clock period longer than it would be if they were in separate stages.
- **Data hazards:** one EX/MEM→WB forwarding path handles both back-to-back ALU dependences and load-use dependences, so neither needs a stall (Section 3.2). The branch comparator is the exception, because it sits a stage earlier than the ALU: it gets its own path from EX/MEM, and a conditional branch or RET still waits one cycle for a value the instruction in front of it has just produced.
- **Control hazards:** every taken branch or jump (BEQ, BGT, B, CALL, RET) costs one bubble, because all of them are resolved in DE/OF. Prediction is static not-taken, so a branch that falls through costs nothing.
- **MDU instructions:** each one keeps EX/MEM busy for L cycles. This is a structural hazard and costs L − 1 stall cycles even when there is no data dependence: 31 cycles with the default Booth multiplier (L = 32) and 32 cycles with the default non-restoring divider (L = 33). With the `default` setting (L = 1) MUL and MULI do not stall at all, which is the floor this hazard is measured against.
- **Clock period:** set by the slower of DE/OF (register read + comparator) and EX/MEM (ALU + data memory in series), so part of the gain from having fewer stages is lost to a longer cycle.

4. The 6-Stage RISC201 Pipeline

The 6-stage pipeline keeps Fetch, Execute, Memory and Writeback as separate stages and splits decode into two: ID, which only decodes the opcode and generates the control signals, and OF, which reads the register file. Each stage now does less, so the critical path of each stage is shorter. The cost is two more pipeline latches (five in total) and a bigger penalty for conditional branches.

The same horizontal control word is used here, read once in ID and passed down the pipeline latches; only the split of its fields across the stages differs from the 4-stage pipeline (Section 6.5).

4.1 Stage Definitions

Stage	Name	What happens
1	IF (Instruction Fetch)	IR ← Mem_instr[PC] and PC ← PC + 4, unless ID (for unconditional jumps) or OF (for conditional branches) redirects the PC.
2	ID (Instruction Decode)	The opcode is decoded, the control word is read, the immediate or offset is sign-extended and the register numbers are latched. The register file is not read yet. B, CALL and RET do not need any register value (their target comes from the offset, or from LR for RET), so they are resolved at the end of this stage and do not wait for OF. An illegal opcode also takes effect here.
3	OF (Operand Fetch)	rs1 and rs2 are read from the register file. BEQ/BGT need these values for the comparison, so they are resolved here, one stage after the unconditional jumps.
4	EX (Execute)	The ALU computes the result, or the address for LD/ST. MDU instructions use the MDU instead and stay in EX for L cycles (Section 2.8).
5	MEM (Memory Access)	Data memory is read (LD, POP) or written (ST, PUSH). Other instructions, including the MDU instructions, pass their result through unchanged.
6	WB (Writeback)	The result is written to the register file (rd, or LR for CALL).

4.2 Instruction Timing

Case 1: independent instructions. Same as before, but each instruction finishes later because there are more stages:

Cycle	1	2	3	4	5	6	7
ADD r3,r1,r2	IF	ID	OF	EX	MEM	WB	
LD r4,0(r5)		IF	ID	OF	EX	MEM	WB

Case 2: dependent ALU instructions, no stall. ADD's result is ready at the end of its EX stage (cycle 4) and SUB enters EX in cycle 5, so the EX/MEM→EX forwarding path is enough. A second path, MEM/WB→EX, handles dependences between instructions that are two apart:

Cycle	1	2	3	4	5	6	7
ADD r3,r1,r2	IF	ID	OF	EX	MEM	WB	
SUB r6,r3,r4		IF	ID	OF	EX	MEM	WB

Case 3: load-use, one stall. Here LD's value is only ready at the end of its separate MEM stage (cycle 5). Without a stall ADD would be in EX in cycle 5, which is one cycle too early. The control unit inserts one bubble so that ADD is in EX in cycle 6, and the value is forwarded from the MEM/WB latch:

Cycle	1	2	3	4	5	6	7	8
LD r3,0(r1)	IF	ID	OF	EX	MEM	WB		
ADD r6,r3,r4		IF	ID	OF	stall	EX	MEM	WB

Case 4: taken conditional branch, two bubbles. BEQ/BGT need both register values, so they can only be resolved at the end of OF (stage 3). By then two more instructions (fetched in cycles 2 and 3) are already in the pipeline, and both are squashed:

Cycle	1	2	3	4	5	6	7
BEQ r1,r2,L	IF	ID	OF	EX	MEM	WB	
(squashed)		IF	squashed				
(squashed)			IF	squashed			
L: ...				IF	ID	OF	EX

Case 5: unconditional jump, one bubble. B and CALL only need the offset and RET only needs LR, and both are available by the end of ID (stage 2). So only the instruction fetched in cycle 2 is squashed:

Cycle	1	2	3	4	5	6	7
B / CALL / RET	IF	ID	OF	EX	MEM	WB	
(squashed)		IF	squashed				
L: ...			IF	ID	OF	EX	MEM

Case 6: branch operand, one or two stalls. As in the 4-stage pipeline the comparator needs its operands a stage before the ALU does, so it has its own forwarding path and a conditional branch waits for the instruction in front of it. Here the wait is one cycle when that instruction is an ALU operation, because its result exists at the end of EX, and two when it is a load or a POP, whose value only exists after MEM:

Cycle	1	2	3	4	5	6	7	8
SUB r3,r1,r2	IF	ID	OF	EX	MEM	WB		
BEQ r3,r0,L		IF	ID	OF	stall	EX	MEM	WB
(squashed)			IF	ID	ID	squashed		
(squashed)				IF	IF	squashed		
L: ...						IF	ID	OF

Case 7: MDU instructions, L − 1 stalls. An MDU instruction stays in EX for L cycles, just as it stays in EX/MEM in the 4-stage pipeline. With L = 4 again, ADD would normally enter EX in cycle 5 but has to wait until cycle 8. By then MUL is in MEM and its result is in the EX/MEM latch, so the existing EX/MEM → EX path forwards it. That is L − 1 = 3 stall cycles, the same as in the 4-stage pipeline:

Cycle	1	2	3	4	5	6	7	8	9	10
MUL r3,r1,r2	IF	ID	OF	EX	EX	EX	EX	MEM	WB	
ADD r6,r3,r4		IF	ID	OF	stall	stall	stall	EX	MEM	WB

4.3 Hazards and Forwarding

- **Data hazards:** since EX and MEM are separate, an ALU result is ready one stage earlier (relative to WB) than a loaded value. EX/MEM → EX and MEM/WB → EX forwarding cover dependent ALU instructions with no stall, but a load-use dependence costs one stall cycle because the loaded value only exists after MEM.
- **Branch operands:** the comparator in OF reads a stage earlier than EX does, so it has its own forwarding path. A conditional branch, or a RET whose LR was just written, waits one cycle behind an ALU producer and two behind a load or a POP (Section 4.2).
- **Control hazards:** a taken conditional branch costs two bubbles (resolved in OF). B, CALL and RET cost one bubble (resolved in ID, as they do not need a register read). Prediction is static not-taken.
- **MDU instructions:** each one keeps EX busy for L cycles, which again means L − 1 stall cycles per instruction, whether or not there is a dependence.
- **Structural:** no stage does two “heavy” things (the ALU and memory are in different stages), so each stage is shorter and the clock can run faster than in the 4-stage design, even though more instructions are in flight.

5. Comparing the 4-Stage and 6-Stage Pipelines

Parameter	4-stage (IF, DE/OF, EX/MEM, WB)	6-stage (IF, ID, OF, EX, MEM, WB)
Pipeline registers	3	5
Clock period	Longer (EX/MEM does ALU + memory in series)	Shorter (each stage does one thing)
Taken conditional branch (BEQ/BGT)	Resolved in DE/OF (stage 2): 1 bubble	Resolved in OF (stage 3): 2 bubbles
Unconditional jump (B/CALL/RET)	Resolved in DE/OF (stage 2): 1 bubble (there is no earlier stage)	Resolved in ID (stage 2): 1 bubble (no register read needed)
Load-use hazard	0 stall cycles (same EX/MEM → WB forwarding as an ALU result)	1 stall cycle (MEM → EX distance)
Back-to-back ALU dependence	0 stall cycles (EX/MEM → WB forwarding)	0 stall cycles (EX/MEM → EX and MEM/WB → EX forwarding)
Branch compares a value the previous instruction just produced	1 stall cycle, whether the producer is an ALU operation or a load (both finish in EX/MEM)	1 stall cycle after an ALU operation, 2 after a load or POP (the comparator sits in OF)

Parameter	4-stage (IF, DE/OF, EX/MEM, WB)	6-stage (IF, ID, OF, EX, MEM, WB)
MDU instruction with an L-cycle MDU	L − 1 stall cycles each (EX/MEM held)	L − 1 stall cycles each (EX held)
Single-cycle multiply (the `default` setting of Section 2.8)	No stall; the multiplier delay competes with ALU + memory for the EX/MEM critical path	No stall; the multiplier delay competes with the ALU for the EX critical path, which reduces the clock advantage
Control unit complexity	Fewer control signal groups per stage, but each stage uses more of them	More stages to control, but each uses a smaller set of signals
Better when	A simple datapath with fewer forwarding/stall cases matters more than clock speed	A higher clock frequency matters more than the larger conditional branch penalty

If we ignore stalls and bubbles, running N instructions takes about (number of stages + N − 1) clock cycles. The 6-stage design has a slightly bigger start-up cost and loses a cycle on every load-use dependence, which the 4-stage design does not. On the other hand, every taken conditional branch costs the 6-stage design one bubble more, while unconditional jumps cost both designs the same single bubble. Which design is faster overall therefore depends on the ratio of the two clock periods, how often a loaded value is used by the very next instruction, how often a branch compares a value the instruction in front of it has just produced, and how often the test programs take conditional branches. Section 5.1 measures all four.

The MDU instructions add the same Σ(L − 1) stall cycles to both designs, so in terms of cycle count they do not favour either one. With the iterative algorithms we build, however, they take up most of the running time of a program that multiplies or divides much, and Section 5.1 shows how far that goes: it flattens the difference between the two pipelines, because a stall both designs pay equally dilutes everything else. Removing those stalls altogether is what the `default` setting measures, and Section 5.1 reports the figure. The cycle count is only half the answer: a single-cycle multiply whose delay exceeded every other stage would set the clock period of both pipelines, and the 6-stage design would lose its clock advantage completely. That is why the `default` rows are read as a floor on cycles, not as a faster processor.

The "control unit complexity" row is not about two encodings: both pipelines read the same unencoded control word, once per instruction, and pass it down their own latches. In the 4-stage pipeline more of its fields are used within the same (longer) stage, and in the 6-stage pipeline the same fields are spread over more stages.

5.1 Measured Results

The figures below come from `make evaluate`, which assembles the five benchmark programs in `tests/bench/`, runs each on both pipelines and reads the simulator's `stats` counters. Each program is written to put one kind of pressure on the datapath, so that a difference between the two designs can be attributed rather than guessed at. Cycle counts are exact and repeatable; they say nothing about clock period, which is the subject of the last column.

Program	What it stresses	Instrs	4-stage cycles	6-stage cycles	4-stage CPI	6-stage CPI	Break-even clock period
loaduse	Every loaded word feeds the instruction straight after it	85	120	153	1.41	1.80	21.6% shorter
branchy	Nested loops, nearly every conditional branch taken	663	1086	1287	1.64	1.94	15.6% shorter
mixed	All four pressures in rough proportion	78	369	386	4.73	4.95	4.4% shorter
arith	A multiply, a divide and a remainder every iteration	113	1652	1665	14.62	14.73	0.8% shorter
calls	Recursive factorial: CALL/RET bubbles and stack traffic	79	357	359	4.52	4.54	0.6% shorter

The 4-stage design wins on cycle count on every program, which is what Sections 3 and 4 predict: it starts up two cycles sooner, it never pays for a load-use dependence, and its conditional branches cost one bubble instead of two. The last column turns that into the only question that matters, because the two designs do not share a clock. The 6-stage pipeline is faster overall only if splitting the stages buys more clock frequency than the table's deficit, and the figure it has to beat is not one number but a range from 0.6% to 21.6% depending on what the program does.

The spread across that range is the useful result. A program dominated by the MDU hides the difference almost entirely: `arith` spends 1512 of its 1652 cycles stalled on the multiplier and divider, and both designs pay those identically, so the 6-stage deficit shrinks to 0.8% and almost any clock improvement would pay for itself. At the other end, `loaduse` is nothing but load-use dependences, which is precisely the hazard the 4-stage design eliminated by merging EX and MEM, and the 6-stage pipeline needs better than a fifth off its clock period to keep up. The two designs are therefore not ordered: which is better depends on the code, and the crossover is reached much sooner on arithmetic-heavy code than on pointer-chasing code.

The per-hazard counters confirm the mechanisms rather than just the totals. On `loaduse` the 6-stage pipeline records 16 load-use stalls against the 4-stage pipeline's 0, one for each of the 16 loads. On `branchy` both record 220 branch-operand stalls, because both comparators sit one stage ahead of the ALU, but the 6-stage pipeline squashes 379 instructions against 199 – close to twice as many, as a two-bubble penalty against a one-bubble penalty should be. On `calls` the 6-stage pipeline also records 16 branch-operand stalls that the 4-stage pipeline avoids entirely: the recursion's base-case test reads a register the arithmetic shortly before the call produced, and the 6-stage comparator sits a stage too early to be handed it without waiting (Section 4.3).

Configuration	Cycles	MDU stall cycles	Adder operations	Note
4-stage, as built	1652	1512	1176	Booth (32) and non-restoring (33)
6-stage, as built	1665	1512	1176	The same stalls, two more stages of start-up
4-stage, shift-and-add	1652	1512	984	The same 32 cycles as Booth; only the adder column moves
6-stage, shift-and-add	1665	1512	984	The same again, with the 6-stage start-up
4-stage, restoring divider	2190	2050	1726	562 restore steps across the 24 divisions
6-stage, restoring divider	2203	2050	1726	The same restores, two more stages of start-up
4-stage, host multiplier	908	768	780	*Bound: `default`, one cycle, no adder of its own*
6-stage, host multiplier	921	768	780	*Bound: the divide and remainder stalls alone*
4-stage, host multiplier and divider	140	0	0	*Bound: no iterative unit at all*
6-stage, host multiplier and divider	153	0	0	*Bound: no iterative unit at all*

The arithmetic benchmark answers the MDU question directly. Of its 1512 stall cycles, 744 are the two multiplies per iteration at 31 stall cycles each and 768 are the divide and remainder at 32 each; handing the multiply to the host removes exactly the former and leaves exactly the latter, at 908 cycles, and handing both over leaves 140. The program's result word is `0x313B9F03` in every one of these runs. That is the shape of every honest experiment here: the arithmetic is the same arithmetic, and only the accounting of time moves. The `default` rows are a bound rather than a design - the host's operator is not an implementable unit - but they are a bound the simulator actually runs, which is what makes them a measurement and not an estimate.

Booth's own contribution is visible in that adder column rather than in the cycles, and building the second multiplier turned the comparison from a claim into a measurement. The 12 iterations of `arith` multiply by 0xFFFFFFFF and by 0x55555555 in turn, passing each as rs2 because that is the operand Booth recodes. Booth

sees the first as one run of 1s and spends a single adder operation on it, and the second as strict alternation, its worst case at 32: 33 for the pair, 396 over the run, which is what the built column shows. Shift-and-add takes the magnitudes first (Section 2.8), so the same two operands cost it the population count of |rs2|: 0xFFFFFFFF is −1, whose magnitude is 1 and whose cost is therefore one adder operation and not 32, while 0x55555555 has 16 bits set. That is 17 for the pair and 204 over the run, and the measured totals are 984 against Booth's 1176.

So on this benchmark shift-and-add is the cheaper multiplier, which is the opposite of what we expected when the row was still a description. The magnitude step is what costs Booth the advantage: a negative multiplier made entirely of 1s is −1, and both recodings reach it in one adder operation. Booth wins where a long run of 1s sits inside a *positive* multiplier, which the magnitude step cannot shorten - 0x7FFFFFFF costs Booth two adder operations against shift-and-add's 31. Neither recoding dominates the other; the operand decides, and that is the result the counter exists to produce. A measurement that confirms the guess every time is not measuring anything.

The divider pair separates in both columns, because restoring spends a cycle as well as an adder operation on every trial subtraction it undoes. Across the 24 divisions of `arith` it restores 562 times, which is 538 cycles and 550 adder operations more than non-restoring, for the same quotients and remainders. Its latency is also the one here that the operands decide rather than the unit - L is 64 − popcount(quotient) - so the pipelines ask the unit what each individual division will cost instead of holding a figure for the instruction. Paying a fixed 33 to avoid a variable 32 + k is the trade non-restoring makes, and on this benchmark it is the right one by a wide margin.

6. Horizontal Microprogrammed Control Unit

6.1 General Organisation

Our control unit does not need the mapping ROM and micro-sequencer that a multi-cycle (non-pipelined) microprogrammed design uses. This is because of one design decision: each instruction is described by exactly one control word, read from the control store once, instead of a microroutine of several words. This works because RISC201 instructions are simple enough that one unencoded word can hold everything an instruction needs in all the stages it passes through. The MDU instructions do not change this, since their L-cycle iteration is counted inside the MDU and not by extra control store reads.

- **Control store:** a ROM with 64 entries, one for every possible 6-bit opcode, each holding the 22-bit control word of Section 6.2. The 37 opcodes in use are filled in according to Section 6.3. The other 27 (0x25–0x3F) all hold a reserved entry that only raises an illegal-opcode exception.
- **Addressing:** since there is one word per instruction, the opcode field of IR is used directly as the control store address. No mapping ROM is needed to turn an opcode into the start address of a microroutine.
- **No micro-sequencer:** no instruction needs a second control store read, so there is no next-address logic (no µPC + 1, no dispatch and no branching inside a microroutine). Section 6.5 explains what we show as the "microPC" in the debugger.
- **Exceptions:** there are three kinds. An *illegal opcode* is raised in decode by the reserved control store entry. A *divide-by-zero* is raised in the execute stage when MDUEn = 1, ALUOp is DIV or MOD, and the divisor (the forwarded rs2 value, or sext(imm) for DIVI/MODI) is 0. A *memory fault* is raised by the data access when the address is not a multiple of 4 or lies outside memory (Section 2.9). In all three cases the faulting instruction and all younger instructions in the pipeline are squashed and all older instructions complete, so exceptions are precise. The simulator reports the first two with the PC of the faulting instruction and the third with the offending address.

6.2 Microword Format

Each group of control lines has its own unencoded field in the microword, so nothing has to be decoded before it drives the datapath. This is what makes the design horizontal: no decoder stands between the control store and the datapath, and the price of leaving every field unencoded is a wider store.

Field	Width	Meaning
RegWrite	1	1: write the destination register (a GPR or LR). 0: leave the register file unchanged.
RegDst	1	0: the destination is the register field decoded from the instruction. 1: the destination is LR (CALL only).
ALUSrcA	1	0: rs1. 1: SP (PUSH/POP only).
ALUSrcB	2	00: rs2. 01: the sign-extended 18-bit immediate/offset. 10: the constant 4 (the SP step for PUSH/POP). 11: reserved.
ALUOp	5	The integer half, bit 4 clear: 00000 ADD, 00001 SUB, 00010 MUL (low 32 bits of the product), 00011 DIV (quotient), 00100 MOD (remainder), 00101 AND, 00110 OR, 00111 XOR, 01000 NOT, 01001 SHL, 01010 SHR, 01011 LUI for MOVHI: shift the immediate left by 14 and fill with zeros, 01100 PASS (MOV), 01101 SRA, 01110–01111 reserved. The floating-point half, bit 4 set: 10000 FADD, 10001 FSUB, 10010 FMUL, 10011 FDIV, 10100 FEQ, 10101 FGT, 10110 ITOF, 10111 FTOI, 11000–11111 reserved. The codes follow the opcode order, and an immediate form uses the same code as its register form (ADDI = ADD, MULI = MUL, and so on) with ALUSrcB = 01. SHR is the logical right shift, which the ISA calls SRL; SRA, the arithmetic one, takes the first code the opcode order leaves over. Five bits rather than four, because fourteen integer codes and eight floating-point ones do not fit sixteen.
MemRead	1	Read the data memory (LD, POP).
MemWrite	1	Write the data memory (ST, PUSH).
MemAddrSrc	1	0: the ALU result is the memory address. 1: SP is used directly as the address (POP only, because POP has to read the old top of stack before SP is incremented).
WBSrc	2	00: execute-stage result (the ALU, or the MDU for MDU instructions, since both drive the same result bus). 01: the value read from data memory. 10: PC + 4 (CALL only). 11: reserved.
SPWrite	1	Write the ALU result into SP (PUSH/POP).
PCSel	2	00: PC + 4 (normal). 01: the branch/jump target from the adder in the decode stage (Sections 3.1 and 4.1). 10: LR (RET). 11: reserved.
CondSel	2	Only used when PCSel = 01. 00: always redirect (B, CALL). 01: redirect if the two registers are equal (BEQ). 10: if the first is greater (BGT).
MDUEn	1	1: send the operands to the MDU instead of the ALU, start it, and hold the execute stage (stalling the stages behind it) until the MDU signals that it is done; ALUOp then chooses the product, quotient or remainder. 0: normal single-cycle ALU operation. MDUEn could be decoded from ALUOp, but in a horizontal word we keep it as a separate line so that the stall logic does not need a decoder.
FpuEn	1	1: send the operands to the FPU instead (Section 2.11), and hold the execute stage for its latency the same way MDUEn does; ALUOp then names the floating-point operation. MDUEn and FpuEn are never both 1. Like MDUEn this line is derivable from ALUOp – it is exactly bit 4 of it – and is kept separate for exactly the same reason.

In total there are 14 fields and 22 bits, so each control store word is 22 bits wide.

6.3 Per-Instruction Control

The table below lists, for each instruction, the destination register, what the ALU or MDU does, what the data memory does, where the writeback value comes from, whether SP is updated and how the PC changes. Together with Section 6.2 this gives the full control word for every opcode. Section 6.4 works out nine of them.

Mnemonic	Reg Dst	ALU / Datapath Op	Memory	WB Src	SP	PC Control
NOP	—	—	—	—	—	—
ADD rd, rs1, rs2	rd	rs1 + rs2	—	ALU	—	—
SUB rd, rs1, rs2	rd	rs1 − rs2	—	ALU	—	—
MUL rd, rs1, rs2	rd	rs1 × rs2, low 32 bits (MDU, L cycles)	—	ALU	—	—

Mnemonic	Reg Dst	ALU / Datapath Op	Memory	WB Src	SP	PC Control
DIV rd, rs1, rs2	rd	rs1 ÷ rs2, quotient (MDU, L cycles)	—	ALU	—	—
MOD rd, rs1, rs2	rd	rs1 mod rs2, remainder (MDU, L cycles)	—	ALU	—	—
AND rd, rs1, rs2	rd	rs1 AND rs2	—	ALU	—	—
OR rd, rs1, rs2	rd	rs1 OR rs2	—	ALU	—	—
XOR rd, rs1, rs2	rd	rs1 XOR rs2	—	ALU	—	—
NOT rd, rs1	rd	NOT rs1	—	ALU	—	—
SHL rd, rs1, rs2	rd	rs1 << rs2[4:0]	—	ALU	—	—
SRL rd, rs1, rs2	rd	rs1 >> rs2[4:0] (logical)	—	ALU	—	—
SRA rd, rs1, rs2	rd	rs1 >> rs2[4:0] (arithmetic)	—	ALU	—	—
ADDI rd, rs1, imm	rd	rs1 + imm	—	ALU	—	—
SUBI rd, rs1, imm	rd	rs1 − imm	—	ALU	—	—
MULI rd, rs1, imm	rd	rs1 × imm, low 32 bits (MDU, L cycles)	—	ALU	—	—
DIVI rd, rs1, imm	rd	rs1 ÷ imm, quotient (MDU, L cycles)	—	ALU	—	—
MODI rd, rs1, imm	rd	rs1 mod imm, remainder (MDU, L cycles)	—	ALU	—	—
LD rd, imm(rs1)	rd	rs1 + imm (address)	Read[ALU addr]	Mem	—	—
ST rs2, imm(rs1)	—	rs1 + imm (address)	Write[ALU addr] ← rs2	—	—	—
MOV rd, rs1	rd	PASS rs1	—	ALU	—	—
MOVHI rd, imm	rd	imm << 14 (zero-filled)	—	ALU	—	—
PUSH rs1 (or LR)	—	SP − 4	Write[ALU addr] ← rs1 or LR	—	ALU → SP	—
POP rd (or LR)	rd or LR	SP + 4	Read[SP]	Mem	ALU → SP	—
CALL offset	LR	—	—	PC+4	—	target (always)
RET	—	—	—	—	—	LR
B offset	—	—	—	—	—	target (always)
BEQ rs1, rs2, offset	—	—	—	—	—	target if EQ
BGT rs1, rs2, offset	—	—	—	—	—	target if GT
FADD rd, rs1, rs2	rd	rs1 + rs2 (FPU, L cycles)	—	ALU	—	—
FSUB rd, rs1, rs2	rd	rs1 − rs2 (FPU, L cycles)	—	ALU	—	—
FMUL rd, rs1, rs2	rd	rs1 × rs2 (FPU, L cycles)	—	ALU	—	—
FDIV rd, rs1, rs2	rd	rs1 ÷ rs2 (FPU, L cycles)	—	ALU	—	—
FEQ rd, rs1, rs2	rd	rs1 == rs2 → 0 or 1 (FPU)	—	ALU	—	—
FGT rd, rs1, rs2	rd	rs1 > rs2 → 0 or 1 (FPU)	—	ALU	—	—
ITOF rd, rs1	rd	int32 to binary32 (FPU)	—	ALU	—	—
FTOI rd, rs1	rd	binary32 to int32 (FPU)	—	ALU	—	—

The eight floating-point instructions all look the same here, and that is the point: to the datapath they are ordinary register writes whose execute stage happens to take several cycles. Only FpuEn and ALUOp tell them apart, and only the FPU knows the operands are not integers. What they report goes to FPSR (Section 2.10).

The L bit of the S format is an instruction field, not a control line: it picks which register supplies the value PUSH stores and which one POP writes, and the control word is the same either way. That is why PUSH and POP have one entry each rather than two.

Opcodes 0x25–0x3F use the reserved control store entry (Section 6.1) and raise an illegal-opcode exception, so no control line is ever left undefined.

6.4 Worked Examples

We picked three instructions that use different parts of the datapath: LD (ALU and memory), CALL (link register and PC redirect) and BEQ (conditional compare). For each one we list every field and then the full 22-bit word, grouped by field in the order of Section 6.2.

Field	LD rd, imm(rs1)	CALL offset	BEQ rs1, rs2, offset
RegWrite	1	1	0
RegDst	0	1	0 (unused)
ALUSrcA	0 (rs1)	0 (unused)	0 (unused)
ALUSrcB	01 (imm)	00 (unused)	00 (unused)
ALUOp	00000 (ADD)	00000 (unused)	00000 (unused)
MemRead	1	0	0
MemWrite	0	0	0
MemAddrSrc	0 (unused)	0 (unused)	0 (unused)
WBSrc	01 (mem)	10 (PC+4)	00 (unused)
SPWrite	0	0	0
PCSel	00	01 (target)	01 (target)
CondSel	00 (n/a)	00 (unconditional)	01 (EQ)
MDUEn	0	0	0
FpuEn	0	0	0
22-bit word (grouped)	1 0 0 01 00000 1 0 0 01 0 00 00 0 0	1 1 0 00 00000 0 0 0 10 0 01 00 0 0	0 0 0 00 00000 0 0 0 00 0 01 01 0 0
22-bit word (hex)	0x220880	0x300110	0x000014

The six MDU instructions all follow one pattern: RegWrite = 1, RegDst = 0, ALUSrcA = 0 (rs1), WBSrc = 00, no memory, SP or PC change (MemRead = MemWrite = MemAddrSrc = SPWrite = 0, PCSel = CondSel = 00) and MDUEn = 1. Only ALUSrcB (00 for rs2, 01 for the immediate) and ALUOp are different:

Instruction	Opcode	ALUSrcB	ALUOp	22-bit word (grouped)	Hex
MUL rd, rs1, rs2	0x03	00 (rs2)	00010 (MUL)	1 0 0 00 00010 0 0 0 00 0 00 00 1 0	0x202002
DIV rd, rs1, rs2	0x04	00 (rs2)	00011 (DIV)	1 0 0 00 00011 0 0 0 00 0 00 00 1 0	0x203002
MOD rd, rs1, rs2	0x05	00 (rs2)	00100 (MOD)	1 0 0 00 00100 0 0 0 00 0 00 00 1 0	0x204002
MULI rd, rs1, imm	0x0F	01 (imm)	00010 (MUL)	1 0 0 01 00010 0 0 0 00 0 00 00 1 0	0x222002
DIVI rd, rs1, imm	0x10	01 (imm)	00011 (DIV)	1 0 0 01 00011 0 0 0 00 0 00 00 1 0	0x223002
MODI rd, rs1, imm	0x11	01 (imm)	00100 (MOD)	1 0 0 01 00100 0 0 0 00 0 00 00 1 0	0x224002

The eight floating-point instructions are the same shape again, with FpuEn = 1 in place of MDUEn and ALUSrcB = 00 throughout, since none of them takes an immediate. Only ALUOp differs, so the control word is the opcode’s ALUOp code shifted into place:

Instruction	Opcode	ALUSrcB	ALUOp	22-bit word (grouped)	Hex
FADD rd, rs1, rs2	0x1D	00 (rs2)	10000 (FADD)	1 0 0 00 10000 0 0 0 00 0 00 00 0 1	0x210001
FSUB rd, rs1, rs2	0x1E	00 (rs2)	10001 (FSUB)	1 0 0 00 10001 0 0 0 00 0 00 00 0 1	0x211001
FMUL rd, rs1, rs2	0x1F	00 (rs2)	10010 (FMUL)	1 0 0 00 10010 0 0 0 00 0 00 00 0 1	0x212001
FDIV rd, rs1, rs2	0x20	00 (rs2)	10011 (FDIV)	1 0 0 00 10011 0 0 0 00 0 00 00 0 1	0x213001
FEQ rd, rs1, rs2	0x21	00 (rs2)	10100 (FEQ)	1 0 0 00 10100 0 0 0 00 0 00 00 0 1	0x214001
FGT rd, rs1, rs2	0x22	00 (rs2)	10101 (FGT)	1 0 0 00 10101 0 0 0 00 0 00 00 0 1	0x215001
ITOF rd, rs1	0x23	00 (unused)	10110 (ITOF)	1 0 0 00 10110 0 0 0 00 0 00 00 0 1	0x216001
FTOI rd, rs1	0x24	00 (unused)	10111 (FTOI)	1 0 0 00 10111 0 0 0 00 0 00 00 0 1	0x217001

6.5 Fitting the Control Unit into the Pipelines

Since the control word is produced once per instruction directly from the opcode, neither pipeline has an incrementing microprogram counter or any dispatch logic. One combinational read of the control store in decode gives the whole control word (Section 6.1).

The assignment still asks for the CLI debugger to show the microPC step by step, so we define it as follows. In every cycle, each busy pipeline stage holds the control word of one instruction, and the control store address of that word is simply the instruction's opcode. The debugger prints one microPC value for each busy stage every cycle, written `opcode:step` – so `12:4` is LD's memory-stage microinstruction in the 6-stage pipeline – up to four values in the 4-stage pipeline and up to six in the 6-stage one. Stepping by one cycle moves each value into the next stage. This is our pipelined equivalent of watching a microPC step through a microroutine. The simulator's `ustore` command prints the whole control store, or one opcode's microroutine stage by stage, and `cycle` prints every busy stage with its microPC and its microinstruction.

The MDU and FPU instructions are the only case where a stage keeps the same microPC for more than one cycle. While the MDU is working, the execute stage shows the same opcode (0x03–0x05 or 0x0F–0x11) for L cycles and the stages behind it stay frozen. So the debugger also prints the MDU iteration count next to the execute stage, for example `EX: 0x03 MUL [MDU 7/32]`, and each single step shows some progress. A divide-by-zero is reported in the first execute cycle, before the count starts. An FPU instruction is shown the same way, as `EX: 0x1D FADD [FPU 2/4]`, with the latencies of Section 2.11.

The fields of the control word are not all used in the same stage:

- **4-stage pipeline:** the control store is read in DE/OF (stage 2). PCSel and CondSel are used right away in DE/OF, because the branch target adder and the comparator are in that stage (Section 3.1). The other fields (RegWrite, RegDst, ALUSrcA/B, ALUOp, MemRead/Write, MemAddrSrc, WBSrc, SPWrite, MDUEn and FpuEn) go through the DE/OF→EX/MEM and EX/MEM→WB latches and are used in EX/MEM and WB. MDUEn and FpuEn are both used in EX/MEM, since both decide how long that stage is held.
- **6-stage pipeline:** the control store is read in ID (stage 2). For unconditional jumps (PCSel = 01 with CondSel = 00 for B and CALL, or PCSel = 10 for RET), PCSel is used in ID itself, since no register read is needed; this is why they resolve early (Sections 4.1 and 4.2). For conditional branches (PCSel = 01 with CondSel = 01, 10 or 11), CondSel is carried one more stage to OF, where the registers are read and compared. That extra stage is why a conditional branch costs one more bubble than an unconditional jump (Section 4.3). The remaining fields continue through EX, MEM and WB, and MDUEn and FpuEn are both used in EX.

Across the stages of either pipeline these field sets partition the microword: every control line is driven by exactly one stage, so no line is left unconnected and none is driven twice.

References

[1] S. R. Sarangi, *Computer Organisation and Architecture*, McGraw Hill Education (India), 2015. (SimpleRisc ISA, multiplier and divider algorithms, microprogramming, pipelining.)

[2] CSC-201 Computer Organization & Architecture, Processor Design Assignment, Autumn 2026.